



INF1103: Programming Foundations with DevOps

Week 5: Data Manipulation



Learning Outcomes

- Understand what data manipulation means.
- Read and process structured data.
- Differentiate between CSV, JSON, and databases.
- Store and retrieve data using JSON files in Python.
- Understand the role of SQL and NoSQL.

Revisiting Lists

- Suppose we store a student's information in a list:

```
student = ["Ali", 75, "Singapore"]
```

Name

Marks

City

```
print(student[1])
```

Value of Marks

```
student = [75, "Ali", "Singapore"]
```

```
print(student[1])
```

?????



Revisiting Lists: Limitations

- We must remember the position.
- The code is not very readable.
- If the order changes, the code breaks.



Solution: Dictionary

Python data structure that stores information as **key-value pairs**, allowing us to access data using meaningful names instead of numeric positions.

Dictionary

- A word and its explanation
- built-in data type that stores data in **key-value** pairs.
- Each **key** in a dictionary **is unique** and maps to a **value**.
 - Example:

ID	Name	Score
001	John	100+
002	John	80
...		

Dictionary

- Each key is separated from its value by a colon (:)
 - `key : value` ,
- The items are separated by commas
- The whole thing is enclosed in curly braces
 - An empty dictionary is `{}`
- Instead of remembering positions, we use meaningful names.

```
student = {  
    "name": "Ali",  
    "mark": 75,  
    "city": "Singapore"  
}
```

```
print(student["mark"])
```



Dictionary

- Keys are unique within a dictionary
- Values may not be unique
- Values can be of any type
 - Strings, numbers, tuples, lists, dictionaries etc.
- Keys must be of an **immutable** data type
 - Strings, numbers, or tuples

Dictionary

- Access
 - `value = students[key]`
- Update and Add
 - `students[key] = value`
 - If a key already exists, the old value will be overwritten by the new value.
 - If it is a new key, the key and value pair will be added to the dictionary
- Delete
 - `del students[key]`
 - `students.clear()`



Python Dictionary Methods

Name	Description
dictionary.keys()	Returns list of dictionary keys
dictionary.values()	Returns list of dictionary values
dictionary.items()	Returns list of dictionary items: (key, value) tuple pairs
dictionary.update(dictionary2)	Adds dictionary2's key-values pairs to the first dictionary



Dictionary

- Better Readability
- Represent Real-World Objects
- Easier Updates
- Easier Data Retrieval

Scale up

- One student:

```
student = {  
    "name": "Ali",  
    "mark": 75  
}
```

- Multiple students

```
students = [  
    {"name": "Ali", "mark": 75},  
    {"name": "Sara", "mark": 88},  
    {"name": "John", "mark": 65}  
]
```

This looks like a small student database, doesn't it?

Scale up

- A dictionary is excellent for storing data while a program is running, but once the program closes, **the data is lost.**
- We therefore need a way to save dictionaries to files



Data Manipulation

Data manipulation involves collecting, storing, organizing, retrieving and modifying data.



Data Manipulation

CRUD Operations:

- Create
- Read
- Update
- Delete

Data Formats: JSON

```
Ali,75  
Sara,88
```

Plain Text

```
student = {  
  "name": "Ali",  
  "mark": 75  
}
```

Dictionary

```
{  
  "name": "Ali",  
  "mark": 75  
}
```

JSON



Why JSON?

- Saved to files
- Sent over the Internet
- Shared between applications
- Stored in NoSQL databases

Data Formats: CSV

```
Ali,75  
Sara,88
```

Plain Text

```
student = {  
  "name": "Ali",  
  "mark": 75  
}
```

Dictionary

```
name,mark  
  
Ali,75  
  
Sara,88
```

CSV



CSV vs JSON

Feature	CSV	JSON
Structure	Rows and columns	Key-value pairs
Data Type Support	Mostly text/tabular	Numbers, strings, arrays, objects, booleans
Nested Data	No	Yes
Readable in Excel	Excellent	Limited
File Size	Smaller	Larger
API Usage	Rare	Very common
Best For	Spreadsheet data	Complex application data



When to use which?

Use **CSV** when:

- Data is tabular.
- You want to open it in Excel.
- Simplicity and smaller file size matter.

Use **JSON** when:

- Data has nested relationships.
- You are building web or mobile applications.
- Data is exchanged through APIs.

CRUD operations in JSON

```
import json
students = [
    {"name": "Ali", "mark": 75},
    {"name": "Sara", "mark": 88}
]
for student in students:
    print(student["name"])

students[0]["mark"] = 80

students.append({
    "name": "John",
    "mark": 65
})

with open("students.json", "w") as f:
    json.dump(students, f)
```

← Create Data

← Read Data

← Update Data

← Add Data

← Save JSON File

Scale up

Imagine:

- 10 students
- 100 students
- 10,000 students

Would JSON files still be convenient?

Problems:

- Large files
- Slow searching
- Many users accessing data simultaneously



Solution: Databases

allow us to store, manage, search, update, and secure large amounts of data more efficiently than using files alone.



Why we need Databases?

- Store data permanently
- Search data quickly
- Update data efficiently
- Handle large amounts of data
- Support many users simultaneously
- Maintain data integrity
- Improve security

Types of Databases

1. Relational Databases (SQL)
2. NoSQL Databases

Types of Databases

1. Relational Databases (SQL)

- Data is stored in **tables** consisting of rows and columns.
- **Examples**
 - PostgreSQL
 - MySQL
 - Microsoft SQL Server
 - Oracle Database
 - SQLite

StudentID	Name	Mark
101	Ali	75
102	Sara	88



Why use Relational DBs

- Structured data
- Relationships between tables
- High data integrity

For example:

- University systems
- Banking systems
- Accounting systems

Postgre SQL

- Relational (SQL) database
- Open source
- Stores data in tables
- Very popular in industry

Spreadsheet



Table



PostgreSQL Database

Postgre SQL: CRUD operations

```
CREATE TABLE Students (  
  student_id INT,  
  name VARCHAR(50),  
  mark INT  
);
```

Table Creation

```
INSERT INTO Students  
VALUES (101, 'Ali', 75);
```

Insert a Value

```
SELECT * FROM Students;
```

Select all

```
DELETE FROM Students  
WHERE student_id = 103;
```

Delete a value

Types of Databases

2. NoSQL Databases

- more flexible and often store data as documents.

- **Examples**

- MongoDB
- CouchDB
- Firebase Firestore
- Cassandra
- DynamoDB

```
{  
  "studentID": 101,  
  "name": "Ali",  
  "mark": 75  
}
```



Why use NoSQL DBs

- Flexible structure
- Easy to scale
- Works well with web and mobile applications

For example:

- Social media
- E-commerce
- Real-time applications

MongoDB

- NoSQL database
- Stores data as JSON-like documents
- Flexible structure

Python Dictionary



JSON File



MongoDB Document

MongoDB: CRUD operations

```
use StudentDB
db.createCollection("students")
```

Table Creation

```
db.students.insertOne(
{
  student_id: 101,
  name: "Ali",
  mark: 75
})
```

Insert a Value

```
db.students.find()
```

Select all

```
db.students.deleteOne(
{
  student_id: 103
})
```

Delete a value

SQL vs NoSQL

SQL	NoSQL
Tables	Documents
Fixed schema	Flexible schema
PostgreSQL	MongoDB
Structured data	Semi-structured data

Task for the day!!

You are a Shopee product administrator managing product information:

1. Display all product names from the product list.
2. Add a new product to the existing product list.
3. Update the stock quantity of an existing product after a sale.
4. Identify products priced above \$30 and display their names.

Do you choose MongoDB or JSON for storing all products?





Takeaway!!

- A Python dictionary is how a program represents data in memory.
- JSON is how we store or exchange that data.
- Databases are how we manage large amounts of such data efficiently

Thanks!!!

